
Data Engineering Assessment

Databricks pipeline | Technical assessment responses

01 Technology choices and rationale

The pipeline runs on Databricks, using Delta tables in Unity Catalog, notebooks, Workflows for orchestration and an Asset Bundle for deployment. In production, I would use Azure Databricks. Raw API files would land in ADLS Gen2, while Bronze, Silver and Gold would be Unity Catalog managed tables, allowing Databricks to handle optimisation and maintenance.

Azure Data Factory could orchestrate the landing step if the team already uses it. I kept orchestration in Databricks Workflows so that the whole pipeline lives in one place. Transformations use SQL; Python handles only the HTTP calls and the run log.

Probing the API first shaped the design: one `/data/readings?date=` call returns every station for a day, approximately 490,000 rows in 14 seconds. A week therefore takes 7 calls instead of 5,531 calls per day.

02 Cloud deployment and data access

The bundle defines an orchestrator, one child job per layer, and dev, test and prod targets that differ only in catalog and schedule. Promotion is therefore a deployment rather than a code change.

Analysts query the Gold star schema through a SQL warehouse or Power BI. Data scientists use notebooks with read access to Silver and Gold, plus their own sandbox. Access is managed through least-privilege Unity Catalog grants for each group, with lineage and auditing also provided by Unity Catalog. Other applications can use the SQL endpoint or Delta Sharing.

03 Testing strategy and failure conditions

The tests/ suite runs every notebook unchanged on local Spark against fixtures that reproduce real API quirks: list and pipe-joined values, missing values, invalid timestamps, duplicate references and a new, unexpected field.

The suite contains 18 tests covering a first run, a rerun with corrected and late readings to verify idempotency, catch-up, a truncated API response and a Bronze count mismatch. The last two must fail explicitly. CI runs these tests, along with lint and bundle reference checks, on every pull request.

At runtime, bad rows are quarantined with a reason, counts are reconciled between layers, and every stage writes to `ops.pipeline_run_log`. This log caught a real bug: catch-up reused the main run's ID. Adding a `-catchup` suffix and checking every partition fixed the issue, and a regression test now covers it.

04 Scaling and performance

Cost grows with the number of readings rather than the number of stations, because ingestion uses one bulk call per day regardless of station count. The three entities run in parallel. The `fact_reading` table is liquid-clustered on date and measure, and each MERGE is restricted to the days in scope. The current run loads approximately 5.1 million readings in roughly 6.5 minutes on Free Edition serverless.

For a large, predictable daily load, I would move scheduled jobs from serverless to job clusters. Their size and autoscaling limits are set in the bundle, giving us control over cost and the ability to scale up or down. Serverless would remain for ad hoc and bursty work, where fast start-up matters more.

If the API became the bottleneck, I would backfill from the daily archive CSVs, move to incremental ingestion with Auto Loader, and parallelise the catch-up fetches.

05 Version control and change protection

The demo code is hosted on GitHub with GitHub Actions. In an Azure setting, I would move it to Azure DevOps Repos and Pipelines. Branch policies would protect main: every change would come from a feature branch through a pull request, and merge only after approval from a senior or principal engineer and a successful CI build.

CI lints the code and checks that every notebook, job reference and %run path in the bundle exists. It then runs the end-to-end tests. Orchestration lives in the bundle, so a job change is reviewed in the same pull request as the code it depends on.

06 Incremental updates and station recovery

Each run re-fetches the last three days because stations can report late. Silver rebuilds those days, keeping the newest copy of each reading, and Gold merges on (measure, timestamp). This overlap does not create duplicates: there are 0 duplicates across 5,118,904 readings.

For a station that is silent for more than 24 hours and then resumes, the catch-up job compares the API's latest reading per measure with our watermark. If the gap begins before the three-day window, it fetches from `since=<watermark>` and loads through the same notebooks. One run recovered 49,623 readings.

Gaps beyond the API's four-week retention are flagged for an archive backfill. Stations that remain silent appear in `gold.vw_stale_measures`, and only active stations are marked as alertable.

07 Development lifecycle, CI/CD and rollback

An Azure DevOps pipeline promotes the same bundle through three environments, each with its own workspace and catalog. A reviewed pull request is merged and deployed to dev. Releases are then deployed to test, which holds production-like data, and run there for one to two weeks.

Changes are promoted to prod every two to four weeks through an approval gate, using databricks bundle deploy -t <target>. Urgent fixes follow the same pipeline on a shorter cycle.

If bad data reaches production, I would pause the schedule and use the run_id on every row, together with the run log, to identify exactly what was written. The affected tables can then be restored using Delta RESTORE TABLE ... TO VERSION AS OF. Landing and Bronze retain the raw data, so the fixed code can replay those days without calling the API again.

08 Production monitoring and silent failures

Job failure emails identify hard failures. The ops.pipeline_run_log records rows in, rows out and rows rejected for every stage of every run.

A silent failure is a run that succeeds but loads the wrong data. Warning signs include fewer readings than usual, a jump in the quarantine rate, an API version change, new fields, or active stations going quiet.

Scheduled SQL alerts check these conditions, comparing daily volume with a rolling baseline and monitoring freshness, quarantine rate, drift warnings and alertable stale stations. The two silent failures most likely with this API, a truncated response and dropped rows, already trigger hard failures.